Project Structure - Mala-Ai-Services

```
Mala-Ai-Services/
  app/
    main.py                         # App factory, lifespan, CORS, error handlers
    core/
      config.py                     # Settings via pydantic-settings
      logging.py                    # structlog setup
      exceptions.py                 # Exception hierarchy + FastAPI handlers
      tracing.py                    # OpenTelemetry / Tracing configuration
    api/
      deps.py                       # DI: get services from app.state
      v1/
        router.py                   # Aggregates endpoint routers
        endpoints/
          health.py                 # GET /health, GET /ready
          capture.py                # POST /capture
          mapping.py                # POST /mapping
          pipeline.py               # POST /pipeline/execute
    schemas/
      common.py                     # ErrorResponse, HealthResponse, etc.
      capture.py                    # CaptureResponse
      mapping.py                    # MappingRequest, MappingResponse
      pipeline.py                   # PipelineConfig, PipelineResponse
    services/
      llm/
        base.py                     # BaseLLMClient, GenerationConfig, LLMResponse
        vertex.py                   # VertexLLMClient (Gemini)
        factory.py                  # create_llm_client()
        openai_client.py            # OpenAI API integration client
        google_studio.py            # Google AI Studio integration
      pdf/
        layout_service.py           # PyMuPDF page extraction
      capture/
        capture_service.py          # OCR orchestration
      mapping/
        base_mapper.py              # Abstract 4-pass framework
        financial_mapper.py         # Concrete financial mapper
        category_mapper.py          # Keyword-to-category matching
        prompts/
          metadata.py               # Pass 1 prompt
          period.py                 # Pass 2 prompt
          statement.py              # Pass 3 prompt + get_statement_prompt()
```

```
        notes.py                # Pass 4 prompt
    pipeline/
      base_step.py              # BasePipelineStep, PipelineContext, StepResult
      registry.py               # StepRegistry
      orchestrator.py           # PipelineOrchestrator
      steps/
        capture_step.py         # Wraps CaptureService
        mapping_step.py         # Wraps FinancialMapperService
    validation/
      schema_validator.py       # JSON schema loader + validator
      schemas/                  # JSON schema files (5 files)
  Dockerfile                    # Multi-stage build
  docker-compose.yml            # Single-service compose
  requirements.txt              # Python dependencies
  .env                          # ALL environment variables documented
  README.md                     # Quick-start guide
```